ВСТУП

З базових типів і типів, що визначаються програмістом, можна визначати похідні типи, до яких відносяться покажчики, масиви, функції, структури та об'єднання. Зазвичай агрегуючий тип включає декілька компонентів. При цьому якщо в якості агрегуючого типу розглядати масив, то всі його компоненти мають той самий тип. У той же час у багатьох задачах є необхідним агрегування даних, що мають різні типи. У цьому випадку на допомогу приходять структури.

Мета роботи – освоєння сфери застосування структур у програмах мовою С++ і практичне відпрацювання структур при розв'язанні задач.

1. ТЕОРЕТИЧНІ ОСНОВИ

1.1. Визначення структурного типу і змінних, що мають цей тип

Структура – це об'єднання в єдине ціле множини пойменованих елементів даних. Якщо в масиві всі компоненти мають одне і те ж ім'я і один і той же тип, то в структурі компоненти мають різні імена і в загальному випадку різний тип.

Структурний тип визначається за допомогою наступного формату:

```
struct ім'я_структурного_типу
        {визначення_елементів};
```

де **struct** – специфікатор структурного типу;

ім'я_структурного_типу – ідентифікатор, довільно обираний програмістом;

визначення_елементів – один або більше описів об'єктів; кожен із цих описів служить прототипом для елементів структур даного структурного типу.

Наприкінці визначення структурного типу обов'язково ставиться крапка з комою.

Наприклад, структура може мати такі компоненти:

- прізвище, ім'я, по батькові (тип **char***);
- рік народження (тип **int**);
- група (тип **char**[7]);

- розмір стипендії (тип **double**).

Для опису такого роду даних можна, наприклад, визначити такий тип:

```
struct type_stud
{
    char* Pib;      // Прізвище, ім'я, по батькові
    int year;                    // Рік народження
    char group[10];                        // Група
    double stipendia;          // Розмір стипендії
};
```

При визначенні структур як елементів даних у мові C++ використовується звичайна форма опису:

```
type_stud stud, *point_stud;
```

Структури можуть бути визначені одночасно з визначенням структурного типу:

struct ім'я_структурного_типу
    {визначення_елементів}
список_структур;

Наприклад, структурний тип `type_stud`, структуру `stud` і покажчик `point_stud` можна визначити в одному операторі:

```
struct type_stud
{
    char *Pib;
    int year;
    char group[7];
    double stipendia;
} stud,*point_stud;
```

Структурний тип можна ввести також за допомогою службового слова **typedef**:

typedef struct
    {визначення_елементів}
позначення_структурного_типу;

У цьому випадку при визначенні структур ім’я структурного типу використовується без службового слова **struct**.

Наприклад, у програмі можуть бути такі визначення:

```
typedef struct
{
    double Re;
    double Im;
} complex;
complex cplx1, cplx2;
```

Структурний тип, який вводиться за допомогою **typedef**, може мати ще одне ім’я, що задається стандартним чином:

typedef struct ім’я1_структурного_типу
{визначення_елементів}
ім’я2_структурного_типу;

У цьому випадку структури можуть визначатися і за допомогою ім’я1_структурного_типу, і за допомогою ім’я2_структурного_типу.

У програмі може бути використаний і безіменний структурний тип, який вводиться разом з визначенням структур:

struct
{визначення_елементів}
список_структур;

Ще один спосіб визначення структурного типу дає препроцесорна директива #**define**:

#**define** ім’я **struct** {визначення_елементів}

Тут ім’я – препроцесорний ідентифікатор, слідом за яким йде так званий рядок заміщення; у файлі, що містить таку директиву в усіх місцях, де зустрінеться зазначене ім’я, воно буде замінено рядком заміщення. У даному випадку здійсниться заміщення рядком

struct {визначення_елементів}

Наприклад, якщо є препроцесорна директива

```
#define TYPE_COMPLEX struct \
{ \
    double Re; \
```

```
    double Im; \
}
```

а в програмі далі зустрінеться визначення виду

```
TYPE_COMPLEX cplx1, cplx2;
```

то це визначення неявно набуває наступний вигляд

```
struct
{
    double Re;
    double Im;
} cplx1, cplx2;
```

Пам'ять під структуру виділяється таким чином, щоб було можливим розміщення всіх її елементів. Так, під структуру `stud` типу `type_stud` має бути виділено 22 байт (4 байти під елемент `Pib` типу **`char`***, 4 байти під елемент `year` типу **`int`**, 10 байт під символьний масив `group[10]` і 8 байт під елемент `stipendia` типу **`double`**). Такий обсяг пам'яті структура займає в тому випадку, коли її елементи слідують у пам'яті один за одним. У деяких випадках між окремими елементами структури можуть розміщуватися вільні байти пам'яті. Розмір пам'яті, що виділяється під структуру можна визначити за допомогою операції **`sizeof`**, яка застосовна як до структурного типу, так і до імені структурного типу:

`sizeof`(ім'я_структурного_типу)
`sizeof`(ім'я_структури)
`(sizeof)` ім'я_структури

Для ініціалізації структури необхідно у визначенні структур після її імені та знаку рівності розмістити у фігурних дужках список початкових значень її елементів, розділених комою:

```
struct complex
{
    double Re;
    double Im;
};
complex ComplZero = {0, 0},ImOne={0, 1}, cplx1;
```

Якщо елементом структури є масив або в програмі визначено масив структур, у загальному випадку при ініціалізації необхідно ставити фігурні

дужки і для масиву, і для структури. Проте не буде помилковою і конструкція, у якій внутрішні дужки опущені:

```
complex val1[2] = {{1.3, 0.2}, {1.2, 0.002}},
        val2[3] = {1.2, 1.3,1.1, 7.8, 0.1, 0.5};
```

На відміну від масивів дозволено присвоювання структур:

```
cplx1 = ComplZero;
```

Для доступу до елементів структури застосовують уточнені імена, які мають такий вигляд:

ім'я_структури . ім'я_елемента_структури

Наприклад, для доступу до елементів визначеної вище структури `ComlZero`, використовуються наступні два уточнених імені – `ComlZero.Re` і `ComplZero.Im`.

Формування уточненого імені здійснюється за допомогою операції «крапка», яка називається операцією доступу до елементу структури (чи об'єднання) і має найвищий пріоритет. Щоб звернутися до елементу структури, що входить до масиву структур, формують уточнене ім'я, у якому індексується перша складова (ім'я масиву). Наприклад, за допомогою введеного раніше структурного типу `complex` можна визначити масив структур:

```
complex ArrStruct[10];
```

Для звертання до елементів структури першого по порядку компонента цього масиву можна використовувати уточнені імена `ArrStruct[0].Re` і `ArrStruct[0].Im`.

Покажчики на структури визначаються аналогічно покажчикам на інші типи даних. Наприклад,

```
complex *pt_stru;
```

Для доступу до елементів структури через покажчик застосовується один з наступних двох способів:

(*покажчик_на_структуру) . ім'я_елементу

або

покажчик_на_структуру–>ім'я_елементу

Наприклад:

```
pt_stru = &ArrStruct[3];
(*pt_stru).Re=1.5;//Аналог ArrStruct[3].Re=12.55;
pt_stru->Im=1.2;  //Аналог ArrStruct[3].Im=12.55;
```

Операція «стрілка» має найвищий пріоритет (разом з операцією «крапка»). Вона забезпечує доступ до елементу, що задається другим операндом, у тій структурі, яка задається першим операндом.

Елементами структур, у свою чергу, можуть бути дані структурного типу. У цьому разі для доступу до елементів вкладеної структури використовують операцію «крапка» або «стрілка» для кожного рівня вкладеності.

1.2. Об'єднання

Деяку подібність із структурами мають об'єднання, які описуються (визначаються) за допомогою службового слова **union**. У всьому іншому формат опису (визначення) об'єднання збігається з форматом опису (визначення) структури. Таким чином, можливі наступні формати визначення об'єднуючого типу:

union ім'я_об'єднуючого_типу
{визначення_елементів};

typedef union
{визначення_елементів}
позначення_об'єднуючого_типу;

typedef union ім'я1_об'єднуючого_типу
{визначення_елементів}
ім'я2_об'єднуючого_типу;

Як і для структур, можливе використання неіменованого об'єднуючого типу, що вводиться разом з визначенням об'єднань:

union
{визначення_елементів}
список_об'єднань;

На відміну від елементів структури, які розміщуються в пам'яті один за одним, елементи об'єднання розташовуються в одній і тій же області пам'яті, накладаючись один на одного. Розмір пам'яті, що відводиться під

об’єднання, визначається розміром його найбільшого за обсягом пам’яті елементу.

Структури можуть бути елементами об’єднань, також як об’єднання можуть бути елементами структур.

Доступ до елементів об’єднання здійснюється так само, як і до елементів структури:

ім’я_об’єднання.ім’я_елементу

(*покажчик_на_об’єднання).ім’я_елементу

покажчик_на_об’єднання->ім’я_елементу

2. ПРИКЛАДИ ПРОГРАМ

Приклад 1. Дано два комплексних числа. Знайти їх добуток.

Розв’язок.

```
#include <iostream>
#include <conio.h>

using namespace std;

int main()
{
    struct complex          // Визначення структурного типу
    {
        float Re;
        float Im;
    };
    complex cplx1, cplx2, mult;      // Визначення структур
    cout << "\nEnter the real and imaginary parte"
            " of the first number\n";
    cin >> cplx1.Re >> cplx1.Im;       // Операція "крапка"
    cout << "\nEnter the real and imaginary parte"
            " of the second number\n";;
    cin >> cplx2.Re >> cplx2.Im;
    mult.Re = cplx1.Re * cplx2.Re - cplx1.Im * cplx2.Im;
    mult.Im = cplx1.Re * cplx2.Im + cplx1.Im * cplx2.Re;
    cout << "The real part: "  << mult.Re <<
            "\nThe imaginary part: "<< mult.Im;
    cout << "\nPress any key to exit.\n";
    _getch();
```

```
    return 0;
}
```

Приклад 2. Дано декілька звичайних дробів (можливо, неправильних і скоротних; знаменник завжди додатний). Знайти добуток цих дробів, записавши його як нескоротній дріб.

Розв'язок.

```
#include <iostream>
#include <conio.h>

using namespace std;

struct TFraction            // Визначення структурного типу
{
    int Numerator;
    int Denominator;
};
   // Найбільший спільний дільник (Greatest Common Divisor)
int GCD( int a, int b);
                            // Добуток двох звичайних дробів
TFraction Multiple(TFraction Fract1, TFraction Fract2);
                                  // Повне скорочення дробу
TFraction Reduce(TFraction Fraction);

int main()
{
    TFraction const One = {1, 1};   // Визначення константи
    TFraction Product = One;        // Визначення змінної і
    TFraction Fractions[] = {{-15, 125}, {-3, 4}, // масиву
                     {-35, 75}, {80, 70}};
    TFraction *PtFraction = &Fractions[0];    // Визначення
         // покажчика на структуру з ініціалізацією адресою
                                // нульового елемента масиву
    int k;
                  // Обчислюємо кількість елементів масиву:
    k = sizeof(Fractions) / sizeof(Fractions[0]);  // обсяг
                   // пам'яті,виділеної під масив, ділиться
                   // на розмір одного (нульового) елемента
              // Нижче демонструється використання покажчика
    for (int i = 0; i < k; i++)
        Product = Multiple(Product, *PtFraction++);
    cout << "Fractions:\n";
```

```
    PtFraction = Fractions;
    for (int i = 0; i < k; i++, PtFraction++)
        cout << PtFraction->Numerator << " / " <<   // Опе-
           PtFraction->Denominator <<    // рація "стрілка"
           endl;
    cout << "Product = "  << Product.Numerator <<
         " / " << Product.Denominator; // Операція "крапка"
    cout << "\nPress any key to exit.\n";
    _getch();
    return 0;
}

TFraction Multiple(TFraction Fract1, TFraction Fract2)
{
    TFraction Fract = Reduce(Fract2);
    Fract2.Numerator = Fract1.Numerator;
    Fract2.Denominator =  Fract.Denominator;
    Fract1.Numerator = Fract.Numerator;
    Fract1 = Reduce(Fract1);
    Fract2 = Reduce(Fract2);
    Fract.Numerator = Fract1.Numerator * Fract2.Numerator;
    Fract.Denominator = Fract1.Denominator *
                        Fract2.Denominator;
    return Reduce(Fract);
}

TFraction Reduce(TFraction Fraction)
{
    int sign = 1;
    if (Fraction.Numerator < 0)
    {
        sign = -1;
        Fraction.Numerator = -Fraction.Numerator;
    }
    int GCDnd = GCD(Fraction.Numerator,
                    Fraction.Denominator);
    Fraction.Numerator /= (GCDnd * sign);
    Fraction.Denominator /= GCDnd;
    return Fraction;
}

// Умова: a >= 0, b >= 0, a != 0 || b != 0
int GCD( int a, int b)
```

```
{
    int r = a;
    if (a < b)
    {
        a = b;
        b = r;
    }
    while (b != 0)
    {
        r = a % b;
        a = b;
        b = r;
    }
    return a;
}
```

Приклад 3. Дано натуральне число n, дійсні числа x_i, y_i, z_i, $i = 1,2,\ldots,n$, що визначають координати матеріальних точок у декартовому просторі, та дійсні числа m_i, $i = 1,2,\ldots,n$, які задають масу відповідних точок. Знайти координати центру мас.

Розв'язок.

```
#include <iostream>
#include <conio.h>

using namespace std;
struct Point                // Визначення структурного типу з
{                                                  // елементами:
    double M;                                       // маса точки
    double XYZ[3];                          // і масив координат
};

int main()
{
    int n;
    cout << "n = ";
    cin >> n;
    Point *Points = new Point[n];
    Point Center = {                // Визначення структури з
        0, {0, 0, 0}                              // ініціалізацією
    };
    for (int i = 0; i < n; i++)
    {
```

```
            cout << "Point number " << i + 1 << endl;
            cout << "M = ";
            cin >> Points[i].M;
            for (int  k = 0; k < 3; k++)
            {
                cout << (k == 0 ? 'X' : (k == 1 ? 'Y' : 'Z'))
                     << " = ";
                cin >> Points[i].XYZ[k];
            }
        }
        for (int i = 0; i < n; i++)
        {
            Center.M += Points[i].M;
            for (int  k = 0; k < 3; k++)
                Center.XYZ[k] += Points[i].XYZ[k] *
                                 Points[i].M;
        }
        cout << "\Coordinstes of the center of mass:\n";
        for (int k = 0; k < 3; k++)
        {
            Center.XYZ[k] /= Center.M;
            cout << (k == 0 ? 'X' : (k == 1 ? 'Y' : 'Z')) <<
                    " = " << Center.XYZ[k] << endl;
        }
        cout << "\nPress any key to exit.\n";
        _getch();
        return 0;
}
```

Приклад 4. Дано натуральне число N ($N \le 10$) і координати двох точок у N-вимірному декартовому просторі. Знайти відстань між цими точками.

```
#include <iostream>
#include <conio.h>

using namespace std;
int main()
{
    struct TPoint3    // Координати в тривимірному просторі
    { // (можна розглядати і одно- або двовимірний простір)
        double X, Y, Z;
    };
```

```
union TPointN_3                              // Об'єднання
{
    TPoint3 P3;          // Точка в тривимірному просторі
    double x[10];         // Точка в просторі розмірності
};                                          // до 10 включно
int n;

cout << "N = ";
cin >> n;
struct  TPoint           // Визначення структурного типу
{
    int N;                          // Розмірність простору
    TPointN_3 Point;                    // Координати точки
};
TPoint P1, P2;
P1.N = P2.N = n;
cout << "Enter the coordinates of point number 1\n";
for (int i = 0; i < P1.N; i++)
{
    cout << "x[" << i + 1 << "] = ";
    cin >> P1.Point.x[i];
}
cout << "Enter the coordinates of point number 2\n";
for (int i = 0; i < P2.N; i++)
{
    cout << "x[" << i + 1 << "] = ";
    cin >> P2.Point.x[i];
}
double dist = 0;
                                        // Загальний випадок
cout << "N = " << n << endl;
for (int i = 0; i < P1.N; i++)
    dist += (P1.Point.x[i] - P2.Point.x[i]) *
              (P1.Point.x[i] - P2.Point.x[i]);
dist = sqrt(dist);
cout << "Distantion = "  << dist << endl;
                           // Особливий випадок (N == 3)
cout << "N = " << n << endl;
dist = (P1.Point.P3.X - P2.Point.P3.X) *
       (P1.Point.P3.X - P2.Point.P3.X) +
       (P1.Point.P3.X - P2.Point.P3.X) *
       (P1.Point.P3.X - P2.Point.P3.X) +
       (P1.Point.P3.X - P2.Point.P3.X) *
```

```
            (P1.Point.P3.X - P2.Point.P3.X);
    dist = sqrt(dist);
    cout << "Distantion = "  << dist << endl;
    cout << "Press any key to exit.\n";
    _getch();
    return 0;
}
```

Приклад 5. Дано множину з *n* точок на декартовій площині, кожна з який має деякий колір, що заданий його назвою. Ввести координати центра множини, а також координати всіх точок, що мають заданий колір.

```
#include <iostream>
#include <conio.h>

using namespace std;

struct Point
{
    double x;
    double y;
};
int main()
{
    struct  ColorPoint
    {
        Point coordinates;
        char color[11];
    };
    int n;
    cout << "Enter the number of points: ";
    cin >> n;
    ColorPoint* clPoints = new ColorPoint[n];
    for (int i = 0; i < n; i++)
    {
        cout << "Enter point coordinates.\n";
        cout << "x = ";
        cin >> clPoints[i].coordinates.x;
        cout << "y = ";
        cin >> clPoints[i].coordinates.y;
        getchar();
        cout << "Enter point color: ";
        gets_s(clPoints[i].color, 10);
```

```
    }
    Point center = { 0, 0 };
    for (int i = 0; i < n; i++)
    {
        center.x += clPoints[i].coordinates.x;
        center.y += clPoints[i].coordinates.y;
    }
    center.x /= n;
    center.y /= n;
    cout << "Center of the set:  x = " <<
            center.x << ", y = " << center.y << endl;
    char color[11];
    cout << "Enter color: ";
    gets_s(color, 10);
    for (int i = 0; i < n; i++)
        if (strcmp(clPoints[i].color, color) == 0)
            cout << "x = " << clPoints[i].coordinates.x <<
                    ", y = " << clPoints[i].coordinates.y <<
endl;
    cout << "Press any key\n";
    _getch();
}
```

3. ВАРІАНТИ ЗАДАЧ

1. Дано натуральне число n і послідовності дійсних чисел $a_1, a_2, \ldots, a_n$; $x_1, x_2, \ldots, x_n$; $y_1, y_2, \ldots, y_n$. Ці дані визначають на координатній площині n квадратів зі сторонами, паралельними координатним осям: a_i – сторона квадрата, (x_i, y_i) – координати центра i-го квадрата, $i = 1, 2, \ldots, n$. Чи існують на координатній площині точки, що належать усім квадратам? Якщо так, то зазначити координати однієї з них.
2. Дано натуральне число n і множину з n точок на площині, що задаються парами дійсних чисел (x_i, y_i), $i = 1, 2, \ldots, n$. Дано також два дійсних числа (u, v), які визначають координати додаткової точки. Знайти номер точки множини, що є:
   а) найближчою до точки (u, v);
   б) найвіддаленішою від точки (u, v).
3. Множина точок задається аналогічно тому, як вона задана в задачі 2 Знайти точку цієї множини, для якої:

а) сума відстаней до інших точок множини мінімальна;
б) сума відстаней до інших точок множини максимальна;
в) відстань до центра множини мінімальна;
г) відстань до центра множини максимальна.

4. Множина точок задається аналогічно тому, як вона задана в задачі 2. Знайти дві різні точки цієї множини, відстань між якими:
   а) мінімальна;
   б) максимальна.
5. Дано дві множини точок *A* і *B*, задані аналогічно тому, як це описано в задачі 2. Множина *A* містить *m* точок, а множина *B* – *n* точок, де *m* і *n* – задані натуральні числа. Знайти відстань між множинами *A* і *B*. Відстань між двома множинами точок визначати як:
   а) відстань між найближчими одна до одної точками цих множин;
   б) відстань між найвіддаленішими точками цих множин;
   в) відстань між центрами цих множин.

   У двох перших випадках знайти також номери і координати точок, за якими визначалася відстань між множинами.
6. Дано натуральне число *n* і масив (або текстовий файл), що містить інформацію про *n* клієнтів банку: прізвище; ім'я; по батькові; адреса (область, район, місто, вулиця, номер будинку, номер квартири); номер кредитної картки; номер рахунку. Сформувати новий масив (текстовий файл), вибравши з вхідного масиву (файлу) елементи відповідно з певним правилом. **Примітка.** Якщо інформація зберігається в текстовому файлі, вважати, що кількість записів *n* не задано.
7. Дано натуральне число *n* і масив (текстовий файл), що містить інформацію про *n* пацієнтів поліклініки: прізвище; ім'я; рік народження; номер телефону (0 за відсутності телефону); домашня адреса (область, район, місто, вулиця, номер будинку, номер квартири); номер медичної карти; група крові. Вивести інформацію про всіх однофамільців, зареєстрованих у цій поліклініці. **Примітка.** Якщо інформація зберігається в текстовому файлі, вважати, що кількість записів *n* не задано.
8. Дано натуральне число *n* і масив (або текстовий файл), що містить інформацію про *n* футбольні команди: назва; місто; кількість зіграних ігор; кількість виграшів, програшів, нічиїх; кількість гравців. Доповнити інформацію кількістю очок (виграш – 3 очки, нічия – 1 очко, програш – 0 очок). Упорядкувати масив (файл) за зменшенням кількості очок. При рівній кількості очок вище повинна розташуватися команда, яка

зіграла менше ігор; при рівності в двох команд кількості очок і кількості ігор більш високе місце повинна зайняти команда з більшою кількістю перемог, а якщо і за цим показником команди виявляться однаковими, розмістити їх в алфавітному порядку за назвою і містом. **Примітка.** Якщо інформація зберігається в текстовому файлі, вважати, що кількість записів *n* не задано.

9. Дано натуральне число *n* і масив (або текстовий файл), що містить інформацію про *n* країн: назва країни; столиця; державна мова; населення (кількість жителів); площа території; грошова одиниця; курс валюти щодо долара; державний лад. Організувати пошук структур (записів) із заданими ознаками (наприклад, вибір структур по заданому діапазону значень площі). **Примітка.** Якщо інформація зберігається в текстовому файлі, вважати, що кількість записів *n* не задано.
10. Дано натуральні числа *m, n* і два масива (або текстових файла)з інформацією про *m* і *n* фільмів: назва; режисер (прізвище, ім'я); країна; рік випуску; вартість (витрати на випуск); дохід. Упорядкувати ці масиви (переписати інформацію з текстових файлів у бінарні файли і впорядкувати їх вміст) по деякому правилу, спільному для обох масивів (файлів). Переписати вміст цих масивів (файлів) у третій масив (текстовий файл), зберігши властивість упорядкованості і виключивши можливе дублювання інформації. **Примітка.** Якщо інформація зберігається в текстових файлах, вважати, що кількість записів *m, n* не задано.
11. Дано натуральне число *n* і масив (або текстовий файл) з інформацією про *n* автомобілів: марка; колір; серійний номер; реєстраційний номер; кількість дверей; рік випуску; ціна. Здійснити упорядкування вмісту масиву (переписати інформацію в бінарний файл і впорядкувати його вміст) по деякому правилу. **Примітка.** Якщо інформація зберігається в текстовому файлі, вважати, що кількість записів *n* не задано.
12. Дано натуральне число *n* і масив (або текстовий файл), що містить інформацію про *n* альбомів вокально-інструментальних груп: назва групи; назва альбому; кількість пісень на диску; рік випуску альбому; фірма-виробник. Упорядкувати вміст масиву (переписати інформацію в бінарний файл і впорядкувати його вміст) за назвами груп в алфавітному порядку. **Примітка.** Якщо інформація зберігається в текстовому файлі, вважати, що кількість записів *n* не задано.
13. Дано натуральне число *n* і масив, що містить інформацію про *n* музичних товарів: носій (грамплатівка, аудіокасета, лазерний диск);

порядковий номер у каталозі; назва; виконавець (або збірка); час звучання; кількість творів; ціна за каталогом. Переписати інформацію в три нових масиви у відповідності з носіями інформації. **Примітка.** Якщо інформація зберігається в текстовому файлі, вважати, що кількість записів *n* не задано.

14. Дано натуральне число *n* і масив (або текстовий файл), що з відомостями про *n* процесорів: назва процесора; тактова частота; кількість ядер; вартість. Переписати ці дані в бінарний файл і впорядкувати вміст цього файлу за зменшенням кількості ядер, а при однаковій кількості ядер – за зменшенням тактової частоти. Вивести дані про процесори, ціна яких менше заданої величини. **Примітка.** Якщо інформація зберігається в текстовому файлі, вважати, що кількість записів *n* не задано.
15. Дано натуральне число *n* масив (або текстовий файл) з *n* датами (число, місяць, рік). Зазначити всі літні дати, а також найбільш ранню дату. **Примітка.** Якщо інформація зберігається в текстовому файлі, вважати, що кількість записів *n* не задано.
16. Маємо каталог на *n* книг і журнальних статей, де *n* – задане натуральне число. Інформація про книгу має такі складові: автори, назва книги, місце виданаая, видавництво, рік видання, кількість сторінок. Складовими інформації про журнальну статтю є: автори, назва статті, назва журналу, номер, рік видання, номери першої і останньої сторінок, де міститься стаття. Вивести інформацію про публікації, видані після 1990 р. (окремо книги і статті). **Примітка.** Якщо інформація зберігається в текстовому файлі, вважати, що кількість записів *n* не задано.
17. Дано інформацію про оцінки, отримані студентами академічної групи під час сесії. Інформація подається в такому вигляді: прізвище студента (рядок не більше 30 символів), форма навчання (бюджетна або контрактна – одна з латинських літер B або C), 5 оцінок (цілі числа 2, 3, 4, 5 або 0, причому 0 відповідає випадку, коли студент не складав відповідний іспит). Має бути також передбачена можливість наявності інформації про середній бал за результатами сесії (дійсне число), а також про розмір стипендії (дійсне число). За умови, що в групі навчається не більше 40 студентів, описати в програмі масив для зберігання згаданої інформації, розрахувати середній бал кожного зі студентів, які склали всі іспити з оцінками, вищими за 2, і виконати нарахування стипендії усім студентам бюджетної форми навчання із середнім балом 4.0 та більше. Базова стипендія становить 600 грн. Студентам, які отримали максимальні бали з усіх іспитів, нараховується

стипендія, на 25 % вища за базову стипендію. Вивести інформацію про студентів, що мають академічні заборгованості, а також інформацію про нарахування стипендій.